

文档编号: WikiBridge-SDS-1.0

WikiBridge

软件设计规格说明书

零代码本地知识库编译与公网发布平台

项目名称: WikiBridge

文档版本: V1.0

编写人员: 项目组

适用阶段: 课程设计原型与系统设计评审

日期: 2026 年 6 月 14 日

文档变更历史记录

序号	变更日期	变更人员	变更内容详情描述	版本
1	2026 年 6 月 14 日	项目组	基于现有项目简介、汇报材料、架构图与用例图，撰写 WikiBridge 软件设计规格说明书初稿。	V1.0

目录

1、引言

1.1 编写目的

软件设计过程包括体系结构设计、用户界面设计、用例设计、构件设计、类设计、数据设计和部署设计。本文档以软件设计规格说明书的形式，对 WikiBridge 零代码本地知识库编译与公网发布平台的设计方案进行系统描述，形成可用于评审、开发分工、测试设计和后续迭代的设计基线。

本文档重点说明系统如何把用户本地素材整理为结构化 Markdown Wiki，如何在本地提供静态浏览和问答能力，以及如何通过 frp 隧道将本地站点安全、低成本地发布到公网。由于当前项目处于描述性文档和课程设计原型阶段，本文档不声明已有代码实现，而是在现有材料基础上补全可指导实现的软件设计方案。

1.2 读者对象

本文档面向以下读者：

- 项目评审人员和课程设计指导教师，用于评估系统设计的完整性、合理性和可实现性。
- 软件设计人员和开发人员，用于理解系统边界、模块职责、关键流程和数据结构。
- 测试人员和质量保证人员，用于设计功能测试、异常测试和端到端验收场景。
- 配置管理人员和运维人员，用于理解本地端、服务器端和公网访问端的部署关系。

1.3 软件项目概述

项目名称为 WikiBridge，定位为零代码本地知识库编译与公网发布平台。项目面向教师、学生、文字工作者、小团队和其他低技术门槛用户，解决个人资料分散、AI 知识库部署复杂、本地知识库难以安全分享的问题。

系统的核心思想是把知识库生成过程类比为软件编译。用户将 Word、PDF、网页摘录、聊天记录、课堂笔记和文本片段放入本地素材目录，随后在用户端 GUI 中主动触发编译。系统调用大模型 Agent 对素材进行阅读、提炼、去重和结构化，生成带有双向链接的 Markdown Wiki 页面，并通过本地 Web Server 渲染为静态站点。用户登录平台账号后，客户端内置或托管的 frpc 向服务器申请公网映射，服务器通过管理 API 和 frps 分配子域名或端口，最终生成可分享的公网 URL。

系统划分为三个端：

- 用户端：知识拥有者使用的 GUI 软件，负责素材箱管理、知识编译、图谱检查、本地静态服务、问答服务和隧道连接。

- 服务器：平台控制面，负责账号认证、Token 鉴权、端口或子域名分配、frps 连接状态、发布记录和基础限速防滥用。
- 消费端：用户的用户通过浏览器访问公网 URL，浏览 Wiki 页面、跳转双向链接，并通过 Chatbox 进行基于知识库上下文的问答。

1.4 文档概述

本文档共分为三大部分。第一部分为引言，说明编写目的、读者对象、项目概述、术语定义和参考资料。第二部分为软件设计约束，说明设计目标、设计原则、运行约束、安全约束和技术约束。第三部分为软件设计，分别描述体系结构、用户界面、用例、类与构件、数据和部署。

1.5 定义

术语	定义
WikiBridge	本项目名称，表示连接本地知识编译和公网分享的桥接平台。
Add	用户将原始资料加入本地素材箱的阶段。
Compile	系统读取新增素材并调用大模型 Agent 生成或更新 Markdown Wiki 的阶段。
Lint	系统检查 Wiki 图谱质量，包括死链接、孤立页面、重复主题和标题不一致的阶段。
Publish	本地静态站点通过 frp 隧道映射到公网并生成可分享 URL 的阶段。
LLM-Wiki	由大模型参与整理、去重、结构化和链接的 Markdown Wiki。
frp	一种内网穿透工具，由 frpc 客户端和 frps 服务端组成。
控制面	服务器负责的账号、鉴权、隧道调度、状态记录和资源控制能力。
数据面	用户知识内容、本地 Wiki 文件、本地索引和问答上下文等业务数据流。

1.6 参考资料

1. 项目根目录 doc.md：零代码内网穿透本地知识库平台项目简介文档。
2. WHU-Beamer/project_report.tex：WikiBridge 阶段汇报材料。
3. WHU-Beamer/uml_architecture.tex：系统架构图源文件。

4. WHU-Beamer/uml_requirements.tex: 系统用例图源文件。
5. 软件设计规格说明书模板: 软件设计规格说明书-空巢老人看护系统.doc。

2、软件设计约束

2.1 软件设计目标和原则

WikiBridge 的软件设计目标是在 3 到 4 周课程设计周期内形成可演示、可解释、可扩展的端到端原型设计。系统应让普通用户不需要编写代码、不需要手动部署复杂服务,即可完成从本地资料收集、知识库编译、图谱维护到公网发布的完整闭环。

系统设计遵循以下原则:

- 本地优先原则: 用户原始资料、Wiki 页面、向量索引和主要知识处理过程保留在本地,服务器不保存用户知识内容。
- 控制面与数据面分离原则: 服务器只承担认证、授权、隧道调度和状态管理,用户端承担知识生成和本地托管。
- 模块化与低耦合原则: 知识编译、静态站渲染、Chatbox 问答、frp 管理和账号认证通过清晰接口协作,便于替换和扩展。
- 主动触发原则: Add、Compile、Lint 和 Publish 都由用户显式操作或确认,避免系统在用户无感知的情况下处理敏感内容。
- 低成本原则: 静态 Wiki 浏览不消耗 Token,只有编译、图谱检查和主动问答时才产生大模型调用开销。
- 可观测原则: 编译进度、错误日志、本地服务状态、隧道连接状态和公网 URL 均应在用户界面中可见。
- 可追踪原则: 功能需求、用例、模块、数据结构和测试场景之间保持可追踪关系。

2.2 软件设计的约束和限制

约束类别	约束说明
运行环境	用户端应面向普通桌面环境设计,优先支持 Windows 和 macOS; 服务器部署在公网 Linux 主机; 消费端使用现代浏览器访问。
网络条件	用户端通常处于内网或 NAT 环境,公网访问依赖 frpc 与 frps 建立隧道; 网络断开时应保留本地浏览能力。

隐私边界	服务器不保存用户文档、笔记、Wiki 页面、向量索引和问答上下文；只保存账号、发布记录和隧道状态。
安全约束	发布入口需要可回收、可限速、可鉴权；Token 不应明文长期暴露在界面或日志中。
成本约束	静态浏览走本地静态页面和隧道转发，尽量避免为普通访问触发大模型调用。
技术约束	当前阶段以原型验证为主，允许选择轻量技术栈；不把某一具体 GUI 框架或大模型供应商写死。
课程周期	设计以可演示闭环为优先，先打通素材到 Wiki、本地服务到公网 URL，再逐步完善体验和边界场景。

3、软件设计

3.1 软件体系结构设计

系统采用三端分层架构，由用户端、服务器和消费端组成。用户端负责知识内容的数据面，服务器负责平台控制面，消费端负责浏览和交互消费。总体架构如图 ?? 所示。

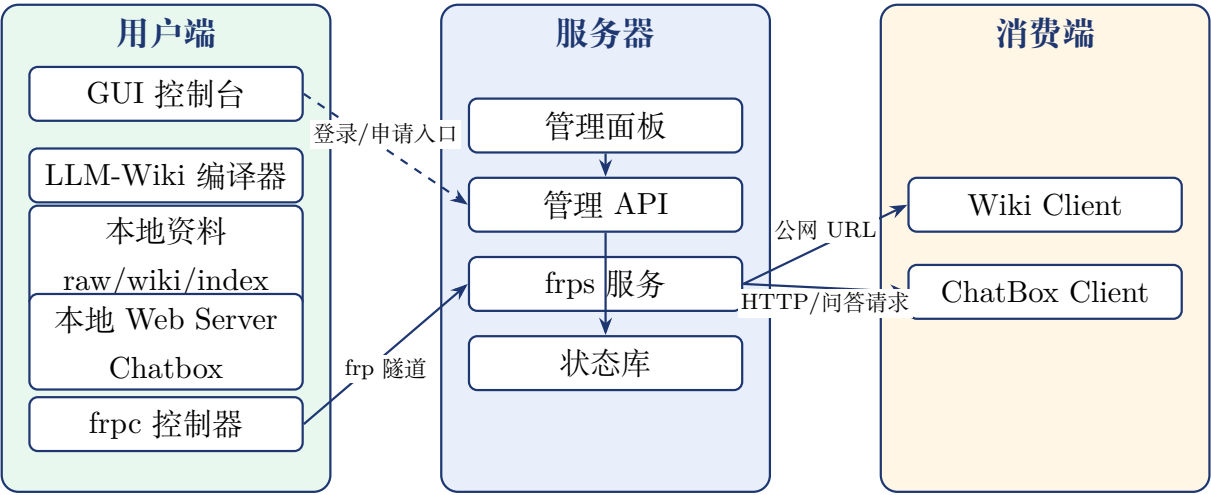


图 1: WikiBridge 系统总体架构图

3.1.1 用户端子系统

用户端是知识拥有者的核心操作入口，承担知识处理和本地托管职责。其内部包含 GUI 控制台、素材箱管理器、Wiki 编译器、Lint 检查器、本地静态 Web Server、Chatbox/检索服务和 frpc 控制器。GUI 控制台屏蔽命令行和网络配置细节，用户只需要选择素材目录、点击编译、查看日志并复制公网 URL。

Wiki 编译器负责读取本地 /raw 目录中的新增素材，调用大模型 Agent 进行阅读、

提炼、去重、结构化和链接生成，并将结果写入 `/wiki` 目录。本地 Web Server 负责渲染 Markdown Wiki 页面和提供基础搜索；本地索引主要用于全文搜索和 Chatbox 问答，不替代长期可读的 Wiki 文本。

3.1.2 服务器子系统

服务器只处理平台控制面，不进入用户知识内容。认证服务负责用户注册、登录和 Token 管理；发布管理 API 负责为用户端分配公网子域名或端口；frps 服务负责接收 frpc 隧道连接并转发外部请求；管理面板提供用户、发布记录、隧道状态和流量统计的查看与管理能力。

3.1.3 消费端子系统

消费端不需要安装软件，外部访问者通过浏览器访问用户分享的公网 URL。静态浏览请求直接访问本地 Web Server 渲染后的 Wiki 页面；当访问者使用 Chatbox 提问时，请求通过隧道转发到用户端本地问答服务，由本地索引和 Wiki 页面上下文支持回答生成，并返回引用来源。

3.2 用户界面设计

用户端 GUI 以任务驱动为核心，主要界面如表 ?? 所示。界面设计目标是让低技术门槛用户无需理解 frp、端口映射、Markdown 渲染和索引构建细节，也能完成知识库编译和发布。

表 4: 用户端主要界面设计

界面	主要元素	职责
欢迎/引导界面	项目简介、数据本地优先说明、开始按钮	解释系统用途和隐私边界，引导用户进入工作台。
登录界面	账号、密码、登录状态、错误提示	完成平台账号登录并获取 Token，用于后续发布入口申请。
工作台主界面	素材箱入口、编译按钮、检查按钮、本地状态、公网 URL	聚合 Add、Compile、Lint 和 Publish 操作，是用户最常用界面。
素材箱界面	<code>/raw</code> 路径、文件列表、导入入口、待处理状态	帮助用户持续加入未处理资料，显示素材是否已被编译。
编译日志界面	进度条、当前任务、日志列表、错误详情	展示大模型处理进度、生成页面数量、失败素材和重试入口。

界面	主要元素	职责
图谱维护界面	死链接、孤立页面、重复主题、修复建议	支持用户查看 Lint 结果，并确认合并、重命名或修复操作。
发布设置界面	本地端口、隧道状态、子域名/端口、访问控制	管理本地服务和公网映射，支持复制 URL、暂停发布和回收入口。
Chatbox 预览界面	问题输入框、回答区域、引用来源、流式状态	让知识拥有者在分享前测试问答效果和引用准确性。

界面跳转关系遵循“登录后进入工作台、工作台进入各任务页面、任务结束后回到工作台”的结构。发生编译失败、网络断开、端口占用或 Token 过期时，系统应在当前界面给出明确状态和可执行处理建议，而不是只输出底层错误日志。

3.3 用例设计

系统主要参与者包括知识库创建者、外部访问者和平台管理员。用例关系如图 ?? 所示。

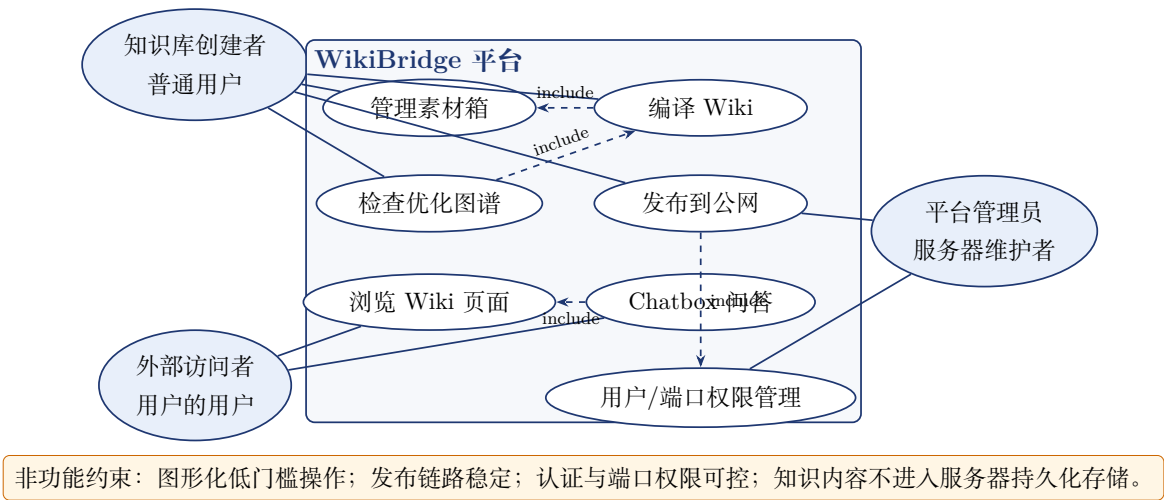


图 2: WikiBridge 用例图

3.3.1 管理素材箱用例

知识库创建者打开素材箱界面，将 Word、PDF、网页摘录、Markdown、TXT 或其他文本材料放入本地 /raw 目录。系统扫描文件基础信息，记录文件路径、更新时间、处理状态和错误状态。若文件暂不支持解析，系统应在素材列表中标记为未处理，并给出可操作提示。

3.3.2 编译 Wiki 用例

用户点击“开始编译”后，系统识别新增或变更素材，调用解析器提取文本，再由大模型 Agent 进行摘要、实体识别、主题聚类、去重合并和页面生成。编译结果写入 /wiki 目录，页面之间使用 [[wiki-links]] 建立双向链接。编译失败的素材不应阻塞已成功生成的页面，系统应记录失败原因并允许重试。

3.3.3 检查优化图谱用例

用户点击“检查优化”后，Lint 检查器扫描 /wiki 目录，识别死链接、孤立页面、重复标题、链接命名不一致和主题过散等问题。系统优先给出建议，涉及删除、合并或大规模重命名的操作需要用户确认。

3.3.4 发布到公网用例

用户登录后，GUI 控制台向服务器管理 API 申请公网入口。服务器校验 Token 后分配子域名或端口，并把配置约束返回给用户端。用户端启动本地 Web Server 和 frpc，建立到 frps 的隧道。发布成功后，界面展示公网 URL、隧道状态、连接时长和暂停入口。

3.3.5 浏览与问答用例

外部访问者打开公网 URL 后浏览静态 Wiki 页面，通过目录、搜索和双向链接跳转。当访问者在 Chatbox 中提问时，请求经 frps 和 frpc 转发到用户端本地问答服务。本地问答服务使用 Wiki 页面和索引作为上下文生成回答，并返回引用来源，便于访问者回到原页面验证。

3.4 核心流程设计

系统主流程由 Add、Compile、Lint 和 Publish 组成，如图 ?? 所示。

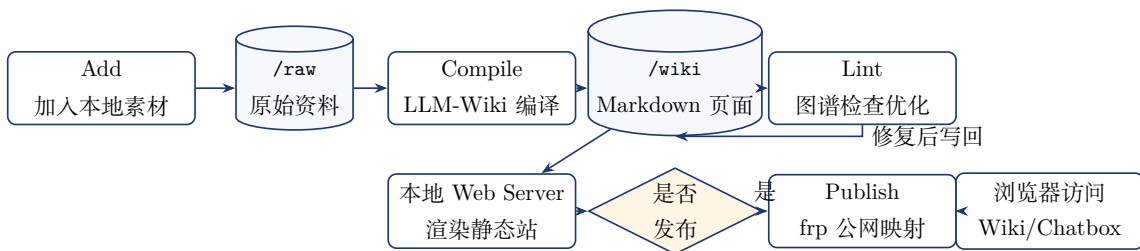


图 3: Add / Compile / Lint / Publish 核心流程图

3.5 类设计和构件设计

当前阶段采用构件级设计结合关键类设计。构件划分如表 ?? 所示。

表 5: 主要构件职责

构件	关键类/对象	职责
GUI 控制台	WorkbenchUI	提供图形化操作入口,展示素材、编译、检查、发布和日志状态。
	TaskLogView	
	PublishStatusView	
账号与认证	AuthManager	管理登录、Token 保存、Token 刷新和退出登录。
	TokenStore	
素材箱管理	RawMaterialManager	扫描 /raw, 维护素材状态, 识别新增、变更和失败文件。
	MaterialRecord	
知识编译	WikiCompiler	调用解析器和大模型 Agent, 生成 Markdown 页面和双向链接。
	AgentOrchestrator	
	WikiPageWriter	
图谱维护	GraphLintEngine	检查死链接、孤立页面、重复主题和命名不一致。
	LinkChecker	
	DuplicateResolver	
本地服务	StaticSiteServer	渲染 Wiki 页面, 提供搜索和基于知识库上下文的问答接口。
	SearchIndexService	
	ChatboxService	
隧道控制	FrpcController	生成 frpc 配置, 启动和监控隧道连接, 回收发布入口。
	TunnelConfig	
	TunnelStatusWatcher	
服务器控制面	AuthService	处理认证、发布入口分配、隧道状态记录和管理面板查询。
	PublishService	
	TunnelAllocator	
	FrpsStatusCollector	

3.5.1 关键类关系

用户端类关系以控制类协调任务、实体类保存状态、边界类展示界面为主。核心关系如下:

- WorkbenchUI 调用 RawMaterialManager、WikiCompiler、GraphLintEngine 和 FrpcController，并订阅任务状态。
- WikiCompiler 依赖 RawMaterialManager 获取待处理素材，依赖 AgentOrchestrator 完成文本理解和结构化，依赖 WikiPageWriter 写入 /wiki。
- GraphLintEngine 读取 /wiki 中的页面和链接关系，输出建议对象，由 WorkbenchUI 展示并等待用户确认。
- FrpcController 依赖 AuthManager 获取 Token，调用服务器 PublishService 获取隧道配置，再启动本地 frpc 进程。
- ChatboxService 依赖 SearchIndexService 获取候选上下文，并将回答引用映射到具体 Wiki 页面。

3.5.2 关键方法设计

类	方法	说明
AuthManager	login(account, password)	调用服务器认证接口，成功后保存 Token 和用户信息。
RawMaterialManager	scanRawDirectory()	扫描素材箱，返回新增、变更、未处理和失败素材。
WikiCompiler	compile(materials)	对待处理素材执行解析、提炼、合并和页面写入。
GraphLintEngine	lint(wikiPath)	输出死链接、孤立页面、重复主题和命名问题列表。
StaticSiteServer	start(port, wikiPath)	启动本地 HTTP 服务并渲染 Wiki 页面。
ChatboxService	answer(question, scope)	基于本地索引和 Wiki 上下文生成回答及引用。
FrpcController	publish(localPort)	申请公网入口、生成配置、启动 frpc 并返回公网 URL。
PublishService	allocateTunnel(userId, localPort)	校验用户权限并分配子域名或端口。

3.6 数据设计

WikiBridge 的数据设计分为本地数据和服务器控制面数据。系统明确避免把用户知识内容上传到服务器持久化保存。

3.6.1 本地数据结构

路径/文件	数据内容	说明
/raw	原始资料	用户放入的 Word、PDF、网页摘录、笔记、Markdown、TXT 等。
/wiki	Markdown Wiki 页面	编译后的结构化页面, 包含标题、正文、标签和 [[wiki-links]]。
/index	本地索引	用于搜索和 Chatbox 的全文索引、向量索引或摘要索引。
config.json	本地配置	素材目录、Wiki 目录、本地端口、模型配置、发布偏好。
material.db	素材状态	文件路径、哈希、更新时间、处理状态、失败原因。
publish.json	发布状态缓存	当前公网 URL、隧道状态、最近连接时间和可回收标识。

3.6.2 服务器数据表

服务器只保存平台控制面数据。建议设计以下数据表：

数据表	字段说明
T_User	id、account、password_hash、display_name、status、created_at、last_login_at。用于保存平台账号，不保存知识内容。
T_AccessToken	id、user_id、token_hash、expire_at、revoked_at。用于 Token 校验和吊销。
T_PublishRecord	id、user_id、subdomain、public_port、local_port、status、created_at、closed_at。用于记录发布入口。
T_TunnelStatus	id、publish_id、online、bandwidth_in、bandwidth_out、connection_count、last_heartbeat。用于管理 frp 状态。
T_RateLimitPolicy	id、user_id、max_bandwidth、max_connections、enabled。用于基础限速和防滥用。

3.6.3 数据安全设计

本地配置中的 Token 应加密保存或使用系统安全存储能力。日志中不得输出完整 Token、用户密码、原始资料全文和模型请求完整上下文。服务器侧密码只保存哈希值，Token

只保存哈希或可验证摘要。发布入口应支持用户主动关闭，管理员也可在滥用或异常流量场景下回收入口。

3.7 部署设计

系统采用分布式部署方式，部署关系如图 ?? 所示。

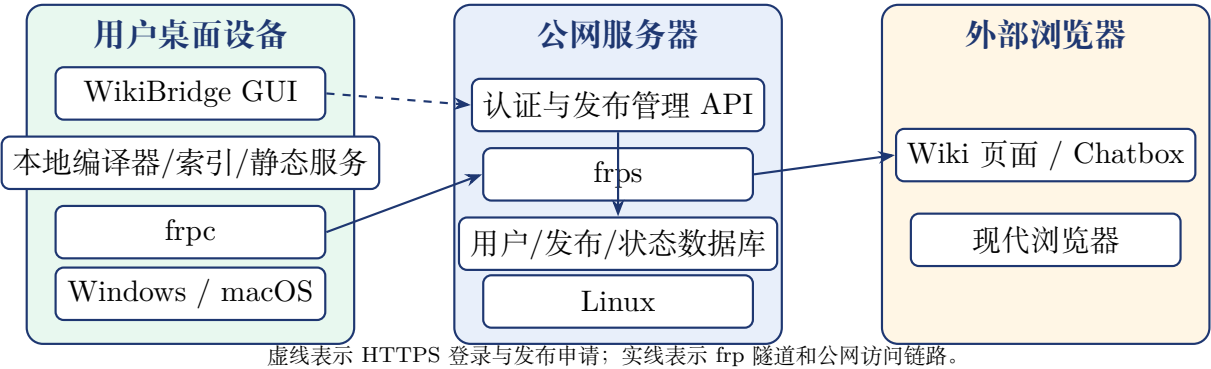


图 4: WikiBridge 部署图

3.7.1 用户端部署

用户端以桌面应用形式部署，安装包内应尽量包含运行所需的 Web Server、frpc、Markdown 渲染器和基础依赖。用户首次启动后选择工作目录，系统在该目录下初始化 /raw、/wiki 和 /index。本地服务默认绑定 127.0.0.1，只有通过 frp 映射后才对外提供访问入口。

3.7.2 服务器部署

服务器部署在具有公网 IP 或公网域名的 Linux 主机上，包含管理 API、管理面板、frps 和数据库。管理 API 与 frps 可分进程部署，通过配置文件或内部接口同步端口与权限约束。数据库只保存平台控制面数据，不保存知识库内容。

3.7.3 消费端部署

消费端无需部署，由外部访问者使用浏览器访问。系统应兼容常见现代浏览器。普通 Wiki 页面浏览应尽量静态化，以降低延迟和成本；Chatbox 请求属于动态能力，应具备超时、错误提示和引用返回。

3.8 异常处理与可靠性设计

异常场景	检测方式	处理策略
------	------	------

素材解析失败	解析器返回错误或文本为空	标记素材失败，不阻塞其他素材编译，允许用户查看原因并重试。
大模型调用失败	超时、限流、返回格式不合法	记录日志，按任务粒度重试，必要时保留半成品并提示用户。
Wiki 链接异常	Lint 扫描死链接或孤立页面	给出修复建议，涉及删除或合并时等待用户确认。
本地端口占用	Web Server 启动失败	自动建议备用端口，并同步更新 frpc 配置。
Token 过期	服务器返回未授权	提示重新登录，暂停发布操作但不影响本地浏览。
隧道断开	frpc 心跳或 frps 状态异常	自动重连并在界面展示状态；多次失败后提示检查网络。
公网访问滥用	流量和连接数超过阈值	服务器限速、临时关闭入口或要求用户重新确认发布。

3.9 测试设计

课程设计原型阶段建议按以下场景验收：

1. 素材箱测试：将多种格式资料加入 /raw，确认系统能识别新增素材并展示状态。
2. 编译测试：触发 Compile 后生成多个 Markdown 页面，页面标题清晰且存在合理双向链接。
3. 图谱检查测试：构造死链接、孤立页面和重复主题，确认 Lint 能识别并给出建议。
4. 本地浏览测试：启动本地 Web Server 后，浏览器可以打开 Wiki 首页、页面详情和搜索结果。
5. 发布测试：登录后申请公网 URL，外部浏览器可通过 URL 访问本地 Wiki。
6. 问答测试：在 Chatbox 中提问，系统返回回答和引用来源，引用可跳转到对应 Wiki 页面。
7. 隐私测试：检查服务器数据库和日志，确认没有保存用户原始资料、Wiki 正文或问答上下文。
8. 异常测试：模拟端口占用、网络断开、Token 过期和模型调用失败，确认界面提示清晰且系统可恢复。

结论

WikiBridge 的设计目标是验证“LLM-Wiki 本地编译 + frp 无感穿透”的端到端可行性。系统通过用户端承担知识内容处理、服务器承担控制面调度、消费端轻量浏览和问答的架构，兼顾低门槛、隐私性、低成本和可演示性。后续实现应优先打通从本地素材到 Markdown Wiki、从本地 Web Server 到公网 URL 的主链路，再逐步完善 Lint 自动修复、访问控制、限速策略和跨平台安装体验。